

Data Structures and Data Frames

Preflight

1. Ensure that RStudio/R is open with the appropriate packages installed (tidyverse)
-

Learning Questions

- This lesson might sound like it's going to be a bit dry: "Why should I care about data types and structures?"
 - **IF YOU UNDERSTAND YOUR DATA, AND YOU UNDERSTAND HOW R SEES YOUR DATA, YOUR ANALYSIS WILL BE MUCH EASIER AND MORE EFFECTIVE**
 - Some questions you might have had before this session may include those on the slide:
 - How can I read data in R?
 - What are the basic data types in R?
 - How do I represent categorical information in R?
 - This session aims to help you answer those
-

Learning Objectives

- In this section, you'll be learning about the data types in R:
 - **WHAT DATA IS**
 - You'll be learning about data *structures*:
 - **WHAT DATA IS BUILT INTO - HOW IT IS ARRANGED**
 - You'll also learn how to find out what type/structure a particular piece of data has
 - Putting it together, you'll see how R's data types and structures relate to the types of data that you work with, yourself.
 - This will help you work much more fluently in R
 - In particular, we'll be working with **DATA FRAMES** - this is the workhorse data structure in R and, when you use R, you will see a *lot* of them.
-

Data Types and Structures in R

- R is **MOSTLY USED FOR DATA ANALYSIS**

- R is set up with key, core data types designed to help you work with your own data
- A lot of the time, R focuses on tabular data.
- **R is very powerful when dealing with tabular data**
- **INTERACTIVE DEMO**
- **SWITCH TO THE CONSOLE**
- Let's start by making a toy dataset.
- We'll eventually save this in your `data/` directory, with the name `feline-data.csv`
- Use the console in **RStudio**
- We'll create a new variable called `cats`

- This holds a `data.frame`

```
cats <- data.frame(coat = c("calico", "black", "tabby"),
                  weight = c(2.1, 5.0, 3.2),
                  likes_catnip = c(1, 0, 1))
```

- We can now save the content of the variable `cats` as a CSV (comma-separated variable) file.
 - We use the `write.csv()` function
 - It is useful to call argument names explicitly so that the code is more readable

```
write.csv(cats, file = "data/feline-data.csv", row.names = FALSE)
```

- We're writing the `cats` data out to file
 - `file` specifies the filename we're writing to
 - `row.names` tells R not to give the rows in the table a name
- **USE THE FILES TAB/VIEW FILES TO VIEW THE CONTENTS OF THE NEW FILE**
- This is a plain text file, and you could have created it using a text editor like Nano or Notepad.
 - The first row is the header row
 - Each individual cat gets its own row
 - Each column is a different kind of data
 - Each column is separated by a comma
- We can load the data from a CSV file in a similar way to how we write it
 - We can use the `read.csv()` function
 - Inspect the dataset by using the variable name

```
cats <- read.csv(file = "data/feline-data.csv")
```

```
cats
```

```
  coat weight likes_catnip
1 calico   2.1           1
2 black   5.0           0
3 tabby   3.2           1
```

- **THINK ABOUT THE DATA TYPES** Are they all the same?
 - **NO** coat is text; weight is some real value (in kg or pounds, maybe), and likes_string looks like it should be TRUE/FALSE but is represented as 1s and 0s
 - **DOES IT MAKE SENSE TO WORK WITH EACH OF THESE ELEMENTS OF DATA AS IF THEY'RE THE SAME THING?** (No)
- Let's explore our dataset
- **EXTRACT A COLUMN FROM A TABLE**
 - Use \$ notation in the console
 - **NOTE THE AUTOCOMPLETION**

```
> cats$weight
```

```
[1] 2.1 5.0 3.2
```

```
> cats$coat
```

```
[1] "calico" "black" "tabby"
```

- **WHAT DID R RETURN?**
- A *vector* (1D ordered collection) of numbers or character strings
- **WE CAN OPERATE ON THESE *VECTORS***
 - *Vectors* are an important concept, and R is largely built so that operations on vectors are central to data analysis.
- Suppose that we discover the scales used to weigh the cats is miscalibrated
 - Every cat weight is underestimated by 2kg
 - We can correct all the weights at once

```
> cats$weight + 2
```

```
[1] 4.1 7.0 5.2
```

- Suppose we want to make a string out of each of the coat types:

```
> paste("My cat is", cats$coat)
```

```
[1] "My cat is calico" "My cat is black" "My cat is tabby"
```

- What if we try to combine these columns?

```
> cats$weight + cats$coat
```

```
Error in cats$weight + cats$coat :  
non-numeric argument to binary operator
```

- **WE HIT AN ERROR**
 - You probably already realised that wasn't going to work, because adding "calico" to "2.1" is nonsense.

- * You already have intuition about this
 - **THESE DATA TYPES ARE NOT COMPATIBLE** for addition
 - R's data types reflect the ways in which data is expected to interact
 - **UNDERSTANDING HOW YOUR OWN DATA MAP TO R's DATA TYPES IS KEY**
 - It's very important to understand how R sees your data (**you want R to see your data the same way you do**)
 - Many problems in R come down to incompatibilities between data and data types.
-

What Data Types Do You Expect?

- **ASK THE STUDENTS**
 - What data types would you expect to see?
 - What data types do you think you would **WANT OR NEED**, from your own experience?
 - **SPEND A COUPLE OF MINUTES ON THIS**
 - The difference between a *data type* and a *data structure*
-

Data Types in R

- R's data *types* are *atomic*: they are **FUNDAMENTAL AND EVERYTHING ELSE IS BUILT UP FROM THEM**, the same way matter is built up from atoms
 - In particular, all the data *structures* are built up from data *types*
- There are only **FIVE DATA TYPES** in R (though one is split into two...) - three that matter here and two we won't deal with
 - **logical**: Boolean, True/False (also 1/0)
 - **numeric**: anything that's a number on the number line; two types of number are supported: **integer** and **double** (real)
 - **character**: text data - readable symbols
- The other two are less relevant to you and we'll not deal with them much today
 - **complex**: complex numbers, defined on the 2D plane
 - **raw**: binary data
- **LET'S LEARN A BIT MORE ABOUT THEM IN THE DEMO**
- We can ask what type of data something is with the `typeof()` function

```
> typeof(cats$weight)
[1] "double"
```

```
> typeof(TRUE)
[1] "logical"
> typeof(3.14)
[1] "double"
> typeof(1)
[1] "double"
```

- This might seem unintuitive: 1 is an integer
 - But R treats all numbers as **doubles** (i.e. “Real” numbers) by default
 - To force R to see a value as an integer, we need to add an L suffix

```
> typeof(1L)
[1] "integer"
> typeof(cats$coat)
[1] "character"
> typeof("Irn Bru")
[1] "character"
```

- No matter how complicated your analysis gets, *all* data in R is interpreted as one of these basic data types.
 - R - like most computer languages - is strict about data types, and this has important consequences.

- **SUPPOSE OUR FRIEND WANTS TO ADD DETAILS OF THEIR OWN CAT TO OUR DATASET**

- We can create a new dataframe to hold this data

```
> additional_cat <- data.frame(coat = "tabby", weight = "2.3 or 2.4", likes_catnip = 1)
> additional_cat
  coat      weight likes_catnip
1 tabby 2.3 or 2.4           1
```

- We can “stack” dataframes using the function **rbind**

```
> cats2 <- rbind(cats, additional_cat)
> cats2
  coat      weight likes_catnip
1 calico        2.1           1
2 black         5           0
3 tabby        3.2           1
4 tabby 2.3 or 2.4           1
```

- Initially, everything looks OK - but we’ve silently caused some problems.
 - Let’s look at the data type of **cats\$weight** in the two dataframes

```
> typeof(cats$weight)
[1] "double"
> typeof(cats2$weight)
[1] "character"
```

- Our friend gave us a different data type in the **weight** column, and this

means we can no longer do the same weight adjustment (adding on 2kg) that we did before:

```
> cats2$weight + 2
Error in cats2$weight + 2 : non-numeric argument to binary operator
```

A Quick Note About Dataframes

- Any column in a dataframe can contain only one datatype.
- Initially, the datatype of `cats$weight` was `double`
- When we added the new cat, the data in that column was `character` - i.e. a string
- Because we can always represent a number as a string, but we can't always represent a string as a number, R chose to *coerce* the datatype from `double` to `character` for that column.
- **INTERACTIVE DEMO**
- We can look at the structure of a dataframe using the `str()` function

```
> str(cats)
'data.frame':   3 obs. of  3 variables:
 $ coat      : chr  "calico" "black" "tabby"
 $ weight    : num  2.1 5 3.2
 $ likes_catnip: int  1 0 1
> str(cats2)
'data.frame':   4 obs. of  3 variables:
 $ coat      : chr  "calico" "black" "tabby" "tabby"
 $ weight    : chr  "2.1" "5" "3.2" "2.3 or 2.4"
 $ likes_catnip: num  1 0 1 1
```

- Here the `str()` function tells us that `cats` and `cats2` are both `data.frames`
 - a very common kind of data structure in R
 - All data frames are composed of rows and columns.
 - Every column has the same number of rows
 - Every column has one and only one data type
 - Each column can be a different data type
- To understand `data.frames`, a bit better, let's meet a different data structure called a *vector*

Vectors

- These are the **MOST COMMON DATA STRUCTURE**
 - Vectors are an ordered collection of data values

- Vectors can contain **ONLY A SINGLE DATA TYPE** (*atomic vectors*)

- **INTERACTIVE DEMO**

- Let's define an **ATOMIC VECTOR OF NUMBERS**

- To create a vector **USE THE c() FUNCTION** (c() is combine; use ?c)

```
> x <- c(10, 12, 45, 33)
> x
[1] 10 12 45 33
```

- Let's check the data type, and what kind of structure we have

```
> typeof(x)
[1] "double"
> length(x)
[1] 4
> str(x)
num [1:4] 10 12 45 33
```

- This is a little cryptic as output
 - We can see from `typeof()` that the vector contains the **double** type
 - The output of `str()` tells us:
 - * it's a **numeric (num)** vector - which includes **double** and **integer** datatypes
 - * there are four elements [1:4] in the vector
 - * the first few datapoint in the vector
- `str()` tells us that the `cats$coat` column is a vector, too:

```
> str(cats$coat)
chr [1:3] "calico" "black" "tabby"
```

Coercion

- **INTERACTIVE DEMO**

- Given what we've done so far, what do you think the following will produce when we use `typeof()`?

- **PAUSE FOR STUDENT SUGGESTIONS**

```
> quiz_vector <- c(2,6,'3')
> typeof(quiz_vector)
[1] "character"
```

- Here, R has enforced that the type of the vector is **character** (string), because we can always represent numbers as strings, but we can't always represent strings as numbers

- **NEXT CALLOUTS**

- This is called *type coercion* and can cause surprises in your code
 - It is a key reason why you need to be aware of the basic data types and how R interprets them.

- **INTERACTIVE DEMO**

- Consider these two vectors

```
> coercion_vector <- c('a', TRUE)
> coercion_vector
[1] "a"      "TRUE"
> another_coercion_vector <- c(0, TRUE)
> another_coercion_vector
[1] 0 1
```

- What do you expect their datatypes to be?

```
> typeof(coercion_vector)
[1] "character"
> typeof(another_coercion_vector)
[1] "double"
```

Coercion

- *Coercion* is what happens when you **CONVERT ONE DATA TYPE INTO ANOTHER**
- If R thinks it needs to, it will **COERCE DATA IMPLICITLY** without telling you
- There is a set order for coercion
 - **logical** can be coerced to **integer**, but **integer** cannot be coerced to **logical**
 - That's because **integer** can describe all **logical** values, but not *vice versa*
 - Everything can be represented as a **character**, so that's the fallback position for R
- **IF THERE'S A FORMATTING PROBLEM IN YOUR DATA, R MIGHT CONVERT THE TYPE TO COPE**
 - R will choose the simplest data type that can represent all items in the vector
- **INTERACTIVE DEMO IN CONSOLE** More useful things to do with vectors
- You can (usually) **COERCE VECTORS MANUALLY** with `as.<type>()`


```

> x
[1] 10 12 45 33
> as.character(x)
[1] "10" "12" "45" "33"
> as.complex(x)
[1] 10+0i 12+0i 45+0i 33+0i
> as.logical(x)
[1] TRUE TRUE TRUE TRUE

```

- Surprising things can happen when R forces one data type into another.
- **If your data doesn't look how you expect it to, *type coercion* may be to blame**
 - Check your data formatting (is there an accidental string/character?)
- Let's look at our `cats` dataframe again

```

> cats
  coat weight likes_catnip
1 calico   2.1           1
2 black   5.0           0
3 tabby   3.2           1
> typeof(cats$likes_catnip)
[1] "integer"

```

- The type of the `likes_catnip` column is recorded as an integer, but we want logical TRUE/FALSE values.
 - We can use the `as.logical()` function to change this

```

> cats$likes_catnip <- as.logical(cats$likes_catnip)
> cats
  coat weight likes_catnip
1 calico   2.1          TRUE
2 black   5.0          FALSE
3 tabby   3.2          TRUE
> str(cats)
'data.frame':   3 obs. of  3 variables:
 $ coat      : chr  "calico" "black" "tabby"
 $ weight    : num  2.1 5 3.2
 $ likes_catnip: logi  TRUE FALSE TRUE

```

Lists

- lists are data structures like *vectors*, **EXCEPT THEY CAN HOLD ANY DATA TYPE**
 - They are not constrained to atomic types
 - They do not coerce their contents' datatypes
- Let's create a new list using the `list()` function

```

> l <- list(1, 'a', TRUE, seq(2, 5))
> length(l)
[1] 4
> l
[[1]]
[1] 1

[[2]]
[1] "a"

[[3]]
[1] TRUE

[[4]]
[1] 2 3 4 5

```

- Individual elements in the list are identified with **DOUBLE SQUARE BRACKETS**
 - There are four elements
 - `[[1]]` is the number 1
 - `[[2]]` is the character "a"
 - `[[3]]` is the logical value TRUE
 - `[[4]]` is the vector of values from 2 to 5
- We can extract a single element from the list with the double square bracket notation

```

> l[[1]]
[1] 1
> l[[4]]
[1] 2 3 4 5

```

- Using the `str()` function we can see the datatypes of all the elements in the list

```

> str(l)
List of 4
 $ : num 1
 $ : chr "a"
 $ : logi TRUE
 $ : int [1:4] 2 3 4 5

```

- The elements of a list can also have names
 - We specify the name when we create the list

```

> l_named <- list(a = "SWC", b = 1:4)
> l_named
$a
[1] "SWC"

```

```
$b
[1] 1 2 3 4
```

- We can use the name of each element to retrieve it, with the `$` notation:

```
> l_named$a
[1] "SWC"
> l_named$b
[1] 1 2 3 4
```

- But it's still a list like any other, and we can use the double square bracket notation.

```
> l_named[[1]]
[1] "SWC"
> l_named[[2]]
[1] 1 2 3 4
> str(l_named)
List of 2
 $ a: chr "SWC"
 $ b: int [1:4] 1 2 3 4
```

Let's Look At A Data Frame

- We didn't go into detail about the `cats` dataframe we created at the start of this episode.
 - But let's look at it more closely

```
> cats
  coat weight likes_catnip
1 calico   2.1         TRUE
2 black    5.0        FALSE
3 tabby    3.2         TRUE
> typeof(cats)
[1] "list"
> cats[[2]]
[1] 2.1 5.0 3.2
> typeof(cats$weight)
[1] "double"
```

- So `cats` is a `list`, and each element in the list is a vector
- But `cats` is a **special kind of list** - a `data.frame` - where all the vectors have the same length.
 - We can see that this is a special kind of list by using the `class()` function

```
> class(cats)
[1] "data.frame"
```

```
> class(l)
[1] "list"
```

- The class `data.frame` represents a standard way of organising data:
 - Each *row* is an observation
 - Each *column* is a variable
 - The data frame represents a series of observations
-

Load Episode Data

- **DEMO IN SCRIPT**
- **DOWNLOAD DATA**
 - Use the link from the Etherpad document
 - Place the file in `data/`
- **CREATE A NEW SCRIPT**
 - Call it `gapminder`
 - Add the code
 - Use `read.table`
 - The data is in CSV format
 - We need to provide a data source (**here, a file**), the separator character, and whether there's a header row

```
# Load gapminder data from a local file
gapminder <- read.table("data/gapminder_data.csv", sep=";", header=TRUE)
```

- **RUN THE SCRIPT** (use Source)
 - **CHECK THE DATA IN THE Environment TAB**
 - Click on `gapminder` in Environment tab.
 - **NOTE COLUMNS**
-

Investigating gapminder

- Now we've loaded our data, let's take a look at it
- **DEMO IN CONSOLE**
 - 1704 rows, 6 columns
 - Investigate types of columns
 - **POINT OUT THAT THE TYPE OF A COLUMN IS INTEGER IF IT'S A FACTOR**
 - **LENGTH OF A DATAFRAME IS THE NUMBER OF COLUMNS**
- It's always useful to get an initial understanding of your data with the `str()` function.

```
> str(gapminder)
'data.frame':  1704 obs. of  6 variables:
```

```

$ country : Factor w/ 142 levels "Afghanistan",...: 1 1 1 1 1 1 1 1 1 1 ...
$ year    : int   1952 1957 1962 1967 1972 1977 1982 1987 1992 1997 ...
$ pop     : num   8425333 9240934 10267083 11537966 13079460 ...
$ continent: Factor w/ 5 levels "Africa","Americas",...: 3 3 3 3 3 3 3 3 3 3 ...
$ lifeExp : num    28.8 30.3 32 34 36.1 ...
$ gdpPercap: num    779 821 853 836 740 ...

```

- The `summary()` function can also be useful
 - With dataframes, this gives a numeric, tabular, or descriptive summary of each column.

```

> summary(gapminder)
      country      year      pop      continent      lifeExp
Afghanistan: 12   Min.   :1952   Min.   :6.001e+04   Africa :624   Min.   :23.60
Albania      : 12   1st Qu.:1966   1st Qu.:2.794e+06   Americas:300   1st Qu.:48.20
Algeria      : 12   Median :1980   Median :7.024e+06   Asia :396   Median :60.71
Angola       : 12   Mean    :1980   Mean    :2.960e+07   Europe :360   Mean    :59.47
Argentina    : 12   3rd Qu.:1993   3rd Qu.:1.959e+07   Oceania : 24   3rd Qu.:70.85
Australia    : 12   Max.    :2007   Max.    :1.319e+09               Max.    :82.60
(Other)      :1632
      gdpPercap
Min.   : 241.2
1st Qu.: 1202.1
Median : 3531.8
Mean    : 7215.3
3rd Qu.: 9325.5
Max.    :113523.1

```

- We can also inspect individual columns of the data.

```

> typeof(gapminder$year)
[1] "integer"
> typeof(gapminder$country)
[1] "integer"

```

- This looks a little weird: why should the country be an integer?
 - Let's investigate further with `str()`

```

> str(gapminder$country)
Factor w/ 142 levels "Afghanistan",...: 1 1 1 1 1 1 1 1 1 1 ...

```

- The country column has been read in as a **factor**
 - A **factor** is a data structure that represents *categorical* data.
 - We can see this in the `str()` output we got, as well: both `country` and `continent` were read in as factors.
 - This is fine, and is what we actually want.
- We can ask about the dimensions of the dataframe with `dim()`

```

> dim(gapminder)
[1] 1704    6

```

- So there are 1704 rows and 6 columns
- What does `length()` produce:

```
> length(gapminder)
[1] 6
```

- Remember that a dataframe is a **list** of **vector** columns, so its length is the number of elements in the list.
 - There are six columns, so the length of the dataframe is 6.
- We can ask specifically for the number of rows and columns, and for other information.
- We can also use `head()` to summarise the dataframe.

```
> nrow(gapminder)
[1] 1704
> ncol(gapminder)
[1] 6
> colnames(gapminder)
[1] "country" "year"      "pop"      "continent" "lifeExp" "gdpPercap"
> head(gapminder)
  country year      pop continent lifeExp gdpPercap
1 Afghanistan 1952 8425333      Asia  28.801  779.4453
2 Afghanistan 1957 9240934      Asia  30.332  820.8530
3 Afghanistan 1962 10267083      Asia  31.997  853.1007
4 Afghanistan 1967 11537966      Asia  34.020  836.1971
5 Afghanistan 1972 13079460      Asia  36.088  739.9811
```

Data Frame Manipulation With dplyr

Learning Objectives

- You're going to **learn to manipulate data.frames with the six *verbs* of dplyr**
 - `select()`
 - `filter()`
 - `group_by()`
 - `summarize()`
 - `mutate()`
 - `%>%` (pipe)
-

What and Why is dplyr?

- dplyr is a package in the **TIDYVERSE**; it exists to enable **rapid analysis of data by groups**
 - For example, if we wanted numerical (rather than graphical) analysis of the `gapminder` data by continent, we'd use `dplyr`
 - It enables group-level analyses without using repetitive code
 - **AVOIDING REPETITION IMPROVES YOUR CODE**
 - More **robust**
 - More **readable**
 - More **reproducible**
-

Split-Apply-Combine

- The **general principle** dplyr supports is called **SPLIT-APPLY-COMBINE**
 - We have a **dataset with several groups in a variable** (column `x`)
 - For example, each patient in our messy data might be a “group”
 - We want to **perform the same operation on each group, independently** - take a mean of `y` for each group, for example
 - So we **SPLIT** the data into groups, on `x`
 - Then we **APPLY** the operation (take the mean for each group)
 - Then we **COMBINE** the results into a new table
-

`select()` - Interactive Demo**

- **DEMO IN CONSOLE**
 - Import dplyr
- ```
> library(dplyr)
```
- The `select()` *verb* **SELECTS COLUMNS**
    - **DEMO IN CONSOLE**
    - If we wanted to select only year, country and GDP data from `gapminder`
    - Specify: **data**, then **columns**
- ```
> head(select(gapminder, year, country, gdpPercap))
  year    country gdpPercap
1 1952 Afghanistan  779.4453
2 1957 Afghanistan  820.8530
3 1962 Afghanistan  853.1007
4 1967 Afghanistan  836.1971
```

```
5 1972 Afghanistan 739.9811
6 1977 Afghanistan 786.1134
```

- Here, we applied a function, but we can also ‘PIPE’ DATA FROM ONE VERB TO ANOTHER
 - These work like pipes in the shell
 - SPECIAL PIPE SYMBOL: %>%
 - Specify only columns

```
> gapminder %>% select(year, country, gdpPercap) %>% head()
  year country gdpPercap
1 1952 Afghanistan 779.4453
2 1957 Afghanistan 820.8530
3 1962 Afghanistan 853.1007
4 1967 Afghanistan 836.1971
5 1972 Afghanistan 739.9811
6 1977 Afghanistan 786.1134
```

filter()

- filter() selects rows on the basis of some condition, or combination of conditions
 - We can use it as a function, with *pipes*
- DEMO IN CONSOLE

```
> head(filter(gapminder, continent=="Europe"))
  country year    pop continent lifeExp gdpPercap
1 Albania 1952 1282697   Europe  55.23  1601.056
2 Albania 1957 1476505   Europe  59.28  1942.284
3 Albania 1962 1728137   Europe  64.82  2312.889
4 Albania 1967 1984060   Europe  66.22  2760.197
5 Albania 1972 2263554   Europe  67.69  3313.422
6 Albania 1977 2509048   Europe  68.93  3533.004
```

- DEMO IN SCRIPT (gapminder.R)
 - One advantage of pipes is that they make chaining *verbs* together
 - MORE READABLE
 - END THE LINES WITH THE PIPE SYMBOL so R knows that there’s a continuation
 - Run the lines and check the output in Environment

```
# Select gdpPercap by country and year, only for Europe
eurodata <- gapminder %>%
  filter(continent == "Europe") %>%
  select(year, country, gdpPercap)
```

Challenge

```
# Select life expectancy by country and year, only for Africa
> afrodata <- gapminder %>%
  filter(continent == "Africa") %>%
  select(year, country, lifeExp)
> nrow(afrodata)
[1] 624
```



Red sticky for a question or issue



Green sticky if complete

Figure 1: images/red_green_sticky.png

group_by()

- The `group_by()` *verb* **SPLITS** `data.frames` **INTO GROUPS ON A VARIABLE/COLUMN PROPERTY**
- **DEMO IN CONSOLE**
 - It returns a **tibble** - a table with extra metadata describing the groups in the table

```
> group_by(gapminder, continent)
# A tibble: 1,704 x 6
# Groups:   continent [5]
   country year    pop continent lifeExp gdpPercap
   <fctr> <int>   <dbl>   <fctr>   <dbl>   <dbl>
1 Afghanistan 1952  8425333   Asia  28.801  779.4453
2 Afghanistan 1957  9240934   Asia  30.332  820.8530
3 Afghanistan 1962 10267083   Asia  31.997  853.1007
4 Afghanistan 1967 11537966   Asia  34.020  836.1971
5 Afghanistan 1972 13079460   Asia  36.088  739.9811
6 Afghanistan 1977 14880372   Asia  38.438  786.1134
7 Afghanistan 1982 12881816   Asia  39.854  978.0114
8 Afghanistan 1987 13867957   Asia  40.822  852.3959
9 Afghanistan 1992 16317921   Asia  41.674  649.3414
10 Afghanistan 1997 22227415   Asia  41.763  635.3414
# ... with 1,694 more rows
```

summarize()

- The combination of `group_by()` and `summarize()` is very powerful

- We can **CREATE NEW VARIABLES** using functions that repeat for each group
- Here, we've split the original table into three groups, and now **CREATE A NEW VARIABLE mean_b THAT IS FILLED BY CALCULATING THE MEAN OF b**
- **DEMO IN SCRIPT**
 - We use the same principle to **calculate mean GDP per continent**

```
> # Produce table of mean GDP by continent
> gapminder %>%
+   group_by(continent) %>%
+   summarize(meangdpPercap=mean(gdpPercap))
# A tibble: 5 x 2
  continent meangdpPercap
  <fctr>      <dbl>
1 Africa      2193.755
2 Americas    7136.110
3 Asia        7902.150
4 Europe     14469.476
5 Oceania    18621.609
```

Challenge 13

- **IN THE SCRIPT**

```
# Find average life expectancy by nation
avg_lifexp_country <- gapminder %>%
  group_by(country) %>%
  summarize(meanlifeExp=mean(lifeExp))
```

- **IN THE CONSOLE**

```
> avg_lifexp_country %>% filter(meanlifeExp == min(meanlifeExp))
# A tibble: 1 x 2
  country      meanlifeExp
  <chr>        <dbl>
1 Sierra Leone    36.8
> avg_lifexp_country %>% filter(meanlifeExp == max(meanlifeExp))
# A tibble: 1 x 2
  country meanlifeExp
  <chr>      <dbl>
1 Iceland    76.5
```



Red sticky for a question or issue



Green sticky if complete

Figure 2: images/red_green_sticky.png

count() and n()

- Two other useful functions are related to `summarize()`
 - `count()` reports a new table of counts by group
 - `n()` is used to represent the count of rows, when calculating new values in `summarize()`
- DEMO IN CONSOLE
- NOTE: standard error is $(\text{std dev})/\sqrt{n}$

```
> gapminder %>% filter(year == 2002) %>% count(continent, sort = TRUE)
# A tibble: 5 x 2
  continent     n
  <fctr> <int>
1 Africa    52
2 Asia     33
3 Europe   30
4 Americas 25
5 Oceania   2
> gapminder %>% group_by(continent) %>% summarize(se_lifeExp = sd(lifeExp)/sqrt(n()))
# A tibble: 5 x 2
  continent se_lifeExp
  <fctr>      <dbl>
1 Africa  0.3663016
2 Americas 0.5395389
3 Asia    0.5962151
4 Europe  0.2863536
5 Oceania 0.7747759
```

mutate()

- `mutate()` CALCULATES NEW VARIABLES (COLUMNS) ON THE BASIS OF EXISTING COLUMNS
- DEMO IN SCRIPT
 - Say we want to calculate the **total GDP of each nation, each year, in \$bn**
 - We'd multiply the GDP per capita by the total population, and divide by 1bn

- **INSPECT THE OUTPUT**

- We have a new data table, which is the `gapminder` data, plus an extra column

```
# Calculate GDP in $billion
gdp_bill <- gapminder %>%
  mutate(gdp_billion = gdpPercap * pop / 10^9)
```

- **WE CAN CHAIN ALL THESE OPERATIONS TOGETHER WITH PIPES**

- We can calculate several summaries in a single `summarize()` command

- We can use the output of `mutate()` in the `summarize()` command

- **DEMO IN SCRIPT**

- We're going to calculate the **total (and standard deviation) of GDP per continent, per year**
- Calculate total GDP first
- Group by continent and year
- Summarise mean and sd of GDP per capita, and total GDP

- **INSPECT THE OUTPUT**

```
# Calculate total/sd of GDP by continent and year
gdp_bycontinents_byyear <- gapminder %>%
  mutate(gdp_billion=gdpPercap*pop/10^9) %>%
  group_by(continent,year) %>%
  summarize(mean_gdpPercap=mean(gdpPercap),
             sd_gdpPercap=sd(gdpPercap),
             mean_gdp_billion=mean(gdp_billion),
             sd_gdp_billion=sd(gdp_billion))
```

```
## ifelse()
```

- **ifelse() IS A FILTER THAT CAN BE USED WITH MUTATE TO CALCULATES NEW VARIABLES (COLUMNS) ON THE BASIS OF EXISTING COLUMNS ONLY IF SOME CONDITION IS MET**

- **DEMO IN SCRIPT**

- Say we want to calculate the total GDP of each nation, each year, in \$bn **BUT ONLY FOR COUNTRIES OVER 10mn PEOPLE**

```
gdp_billion_large_countries <- gapminder %>%
  mutate(gdp_billion_large = ifelse(pop > 10e6,
                                     gdpPercap * pop / 10^9,
                                     NA))
```

- **INSPECT THE OUTPUT**

- We have a new data table, which is the `gapminder` data, plus an extra column

- Similarly, we can scale GDP only in those cases where life expectancy for a nation is over 40

```
gdp_future_bycontinents_byyear_high_lifeExp <- gapminder %>%
  mutate(gdp_futureExpectation = ifelse(lifeExp > 40, gdpPercap * 1.5, gdpPercap)) %>%
  group_by(continent, year) %>%
  summarize(mean_gdpPercap = mean(gdpPercap),
             mean_gdpPercap_expected = mean(gdp_futureExpectation))
```

Tidy Data

Why Tidy Data?

- Data cleaning/processing is **not just a first step** - it must be repeated many time over the course of an analysis
 - new data, new ideas, etc. turn up as you're working
 - About **80% of the effort of data analysis** is cleaning and preparing data for analysis
 - The principles of **tidy data** provide a standard way to organise data values within a dataset
 - “Tidy datasets are all alike, but every messy dataset is messy in its own way”
-

An Untidy Dataset (1)

- Here's a dataset like you might receive it from a colleague
 - It's a **RECTANGULAR TABLE**
 - Made up of **ROWS** and **COLUMNS**, just like the data you've been working with
 - Each row describes a treatment
 - Each column gives the results for a different individual, for each treatment
 - Thinking about how our dataframes are structured, there should be one row per observation.
 - But here the observations are the people
 - So maybe we could transpose the rows and columns so that the people are the rows?
-

An Untidy Dataset (2)

- So we've transposed the rows and columns of the table
- The **data is the same**
- The **layout is different**
- Now we have one row per observation, and we have one column per variable (treatments A and B), and that's what we want, isn't it?
- But the data doesn't have to be structured this way.
 - In fact, this isn't a very good way to structure data for many analyses.
- To understand what this means, **AND WHY IT IS UNTIDY DATA**, we need to consider some data semantics.

Data Semantics

- We need to define three terms
- A dataset, like the one shown, is a collection of **VALUES**
- Each value belongs to a variable, and to an observation
- A **VARIABLE** is something that *can change or vary*
 - They may be values that measure the same underlying attribute, such as height, temperature, or some kind of output result
 - They may be experimental conditions under a researcher's control, such as a treatment, how long that treatment is applied, or other experimental settings
- An **OBSERVATION** is a collection of **values** measured across all **variables** for the same individual or unit
 - The unit is often a person, a collective group like a religion or company, or a physical item like a reactor vessel

Challenge (2min)

- So for our first messy dataset, how would you describe the rows and columns of the table?
 - Are they observations, or variables, or neither?
- **THE ROWS AND COLUMNS IN THE MESSY DATA ARE NEITHER OBSERVATIONS NOR VARIABLES**

A Tidy Dataset (1)

- The dataset contains 18 values:
 - six observations of three variables

- The variables are:
 - **PERSON**: John, Mary and Jane
 - **TREATMENT**: A or B
 - **RESULT**: NA, 16, 3, 2, 11, 1
 - Each **OBSERVATION** includes values for all three variables
-

A Tidy Dataset (2)

- Here is the dataset represented in tidy form
 - You can see each variable has its own column
 - Each observation has one value per column
 - **Note: Missing data counts as a value**
-

Tidy Data

- Tidy data is a **STANDARD** way of structuring a dataset, but it is not the only way, or always the best way
 - It does make it easy to extract the variables you need
 - It is also very well suited to R because it supports vectorisation (which you saw earlier)
- In Tidy Data:
 - Each variable forms a column
 - Each observation forms a row
- Most of the data you receive is unlikely to be Tidy, so you'll probably need to clean it
- And **MESSY DATA CAN BE USEFUL**
 - If your design is completely crossed: e.g. every individual tries every medicine
 - * Messy Data (rows=patients, columns=medicines) is compact
 - Also useful if matrix operations are appropriate
- **INTERACTIVE DEMO**
 - Show that the `gapminder` data is in an intermediate format
 - Three ID variables (continent, country, year)
 - Three observation variables (pop, lifeExp, gdpPercap)
 - This intermediate form can be preferable; there is no advantage to having a single “observation” column, here
- **IN THE CONSOLE**
 - Each column in the `gapminder` dataset is a variable
 - Each row is a set of values: one per variable, comprising a single observation for a combination of country and year
- **WE CAN USE DPLYR DATA METHODS ON THIS DATASET**

```
> head(gapminder)
  country year      pop continent lifeExp gdpPercap
1 Afghanistan 1952  8425333      Asia  28.801  779.4453
2 Afghanistan 1957  9240934      Asia  30.332  820.8530
3 Afghanistan 1962 10267083      Asia  31.997  853.1007
4 Afghanistan 1967 11537966      Asia  34.020  836.1971
5 Afghanistan 1972 13079460      Asia  36.088  739.9811
6 Afghanistan 1977 14880372      Asia  38.438  786.1134
```

Long v Wide

- We often refer to datasets as being “long” or “wide”
 - **LONG datasets have one row per observation, and one column per variable**
 - **WIDE datasets might have multiple arrangements**
 - Wide datasets tend to be easier for humans to read
 - Quite a lot of the time, we tend to record and share datasets in wide format.
 - **BUT MANY R FUNCTIONS ARE DESIGNED TO WORK WITH LONG DATA**
 - Some plotting functions work better with wide data, though
 - The purely long format can require additional grouping operations (like `group_by()`), and it can be more convenient to have an intermediate form like the `gapminder` data.
-

gapminder Wide Dataset

- We’ve been using the nicely-formatted `gapminder` data to make our lives easier.
- **THE DATA YOU RECEIVE IN A RESEARCH CONTEXT WILL NOT USUALLY BE SO NICELY-FORMATTED**
- Let’s load in the wide-format `gapminder` data and take a look at it.

```
> gap_wide <- read.csv("data/gapminder_wide.csv", stringsAsFactors = FALSE)
> str(gap_wide)
'data.frame':   142 obs. of  38 variables:
 $ continent      : chr  "Africa" "Africa" "Africa" "Africa" ...
 $ country        : chr  "Algeria" "Angola" "Benin" "Botswana" ...
 $ gdpPercap_1952: num   2449 3521 1063 851 543 ...
 $ gdpPercap_1957: num   3014 3828 960 918 617 ...
 $ gdpPercap_1962: num   2551 4269 949 984 723 ...
```



```

$ gdpPercap_1967: num 3247 5523 1036 1215 795 ...
$ gdpPercap_1972: num 4183 5473 1086 2264 855 ...
$ gdpPercap_1977: num 4910 3009 1029 3215 743 ...
$ gdpPercap_1982: num 5745 2757 1278 4551 807 ...
$ gdpPercap_1987: num 5681 2430 1226 6206 912 ...
$ gdpPercap_1992: num 5023 2628 1191 7954 932 ...
$ gdpPercap_1997: num 4797 2277 1233 8647 946 ...
$ gdpPercap_2002: num 5288 2773 1373 11004 1038 ...
$ gdpPercap_2007: num 6223 4797 1441 12570 1217 ...
$ lifeExp_1952 : num 43.1 30 38.2 47.6 32 ...
$ lifeExp_1957 : num 45.7 32 40.4 49.6 34.9 ...
$ lifeExp_1962 : num 48.3 34 42.6 51.5 37.8 ...
$ lifeExp_1967 : num 51.4 36 44.9 53.3 40.7 ...
$ lifeExp_1972 : num 54.5 37.9 47 56 43.6 ...
$ lifeExp_1977 : num 58 39.5 49.2 59.3 46.1 ...
$ lifeExp_1982 : num 61.4 39.9 50.9 61.5 48.1 ...
$ lifeExp_1987 : num 65.8 39.9 52.3 63.6 49.6 ...
$ lifeExp_1992 : num 67.7 40.6 53.9 62.7 50.3 ...
$ lifeExp_1997 : num 69.2 41 54.8 52.6 50.3 ...
$ lifeExp_2002 : num 71 41 54.4 46.6 50.6 ...
$ lifeExp_2007 : num 72.3 42.7 56.7 50.7 52.3 ...
$ pop_1952 : num 9279525 4232095 1738315 442308 4469979 ...
$ pop_1957 : num 10270856 4561361 1925173 474639 4713416 ...
$ pop_1962 : num 11000948 4826015 2151895 512764 4919632 ...
$ pop_1967 : num 12760499 5247469 2427334 553541 5127935 ...
$ pop_1972 : num 14760787 5894858 2761407 619351 5433886 ...
$ pop_1977 : num 17152804 6162675 3168267 781472 5889574 ...
$ pop_1982 : num 20033753 7016384 3641603 970347 6634596 ...
$ pop_1987 : num 23254956 7874230 4243788 1151184 7586551 ...
$ pop_1992 : num 26298373 8735988 4981671 1342614 8878303 ...
$ pop_1997 : num 29072015 9875024 6066080 1536536 10352843 ...
$ pop_2002 : int 31287142 10866106 7026113 1630347 12251209 7021078 15929988 4048013
$ pop_2007 : int 33333216 12420476 8078314 1639131 14326203 8390505 17696293 4369038

```

- **ALSO VIEW IN RSTUDIO**

- This is very wide data and it looks like it would be a bit of a nightmare to do the kinds of analyses we have been doing, with the data in this format.

Pivot: Wide to Long

- We want to convert this wide format that's difficult to work with to our nice, longer, intermediate layout.
- **TO DO THIS WE USE THE `pivot_longer()` FUNCTION FROM `dplyr/tidyverse`**

- This makes datasets longer by increasing the number of rows and decreasing the number of columns
- You may be familiar with “pivot tables” from Excel, which are similar.
- It works like the image shows
 - We define the dataset we want to pivot
 - We say which columns we want to convert from wide to long format in the `cols` variable
 - * The `pivot_longer()` function effectively splits the table by these columns
 - * The column name then gets converted to its own, new column, but the values are kept
 - We specify the names of the new “name” and “value” columns with `names_to` and `values_to`.
- For the `gapminder` data, we want to pivot all columns that start with `pop`, `lifeExp` or `gdpPerCap`
 - We’ll put the column names into a column called `obstype_year`
 - We’ll put the values into a column called `obs_values`

```
> gap_long <- gap_wide %>%
+   pivot_longer(
+     cols = c(starts_with('pop'), starts_with('lifeExp'), starts_with('gdpPerCap')),
+     names_to = "obstype_year", values_to = "obs_values"
+   )
> str(gap_long)
tibble [5,112 × 4] (S3: tbl_df/tbl/data.frame)
 $ continent   : chr [1:5112] "Africa" "Africa" "Africa" "Africa" ...
 $ country     : chr [1:5112] "Algeria" "Algeria" "Algeria" "Algeria" ...
 $ obstype_year: chr [1:5112] "pop_1952" "pop_1957" "pop_1962" "pop_1967" ...
 $ obs_values  : num [1:5112] 9279525 10270856 11000948 12760499 14760787 ...
```

- This gives us a long format table with four columns
- **BUT IT’S NOT QUITE WHAT WE WANT, BECAUSE WE HAVE TWO KINDS OF DATA COMBINED IN THE `obstype_year` COLUMN**
 - We want to split `year` into its own column.

`separate()`

- **THE `separate()` FUNCTION SPLITS THE CONTENTS OF A SINGLE COLUMN INTO MULTIPLE NEW COLUMNS**
 - We want to split the year information from every cell in the `obstype_year` column into its own new column.
- The `separate()` function needs to know:
 - dataframe it’s working on
 - the column it’s splitting
 - the names of the new columns it’s making
 - the character it’s splitting the cell content on

- We split the `obstype_year` column into two columns called `obs_type` and `year`, on the underscore separator `"_"`.

```
> gap_long <- gap_long %>% separate(obstype_year, into = c('obs_type', 'year'), sep = "_")
```

- As the year at this point is still a character string, we change the datatype with `as.integer` and take a look at the new dataframe.

```
> gap_long$year <- as.integer(gap_long$year)
> str(gap_long)
tibble [5,112 × 5] (S3: tbl_df/tbl/data.frame)
 $ continent : chr [1:5112] "Africa" "Africa" "Africa" "Africa" ...
 $ country   : chr [1:5112] "Algeria" "Algeria" "Algeria" "Algeria" ...
 $ obs_type  : chr [1:5112] "pop" "pop" "pop" "pop" ...
 $ year      : int [1:5112] 1952 1957 1962 1967 1972 1977 1982 1987 1992 1997 ...
 $ obs_values: num [1:5112] 9279525 10270856 11000948 12760499 14760787 ...
```

- **THIS IS STILL NOT QUITE WHERE WE WANT IT**
 - There are three types of observation combined in `obs_type` and `obs_values` and it would be convenient to split them into new columns

Pivot: Long to Wide

- We want to convert this long format into a slightly wider intermediate layout, for convenience.
- There is a companion function to `pivot_longer()` called `pivot_wider()` that we use for this.
 - It's essentially the reverse of `pivot_longer()`
- **THE `pivot_wider()` FUNCTION SPLITS THE TABLE UP BY VALUES IN THE COLUMN OF “NAMES”**
 - It then changes the name of the “VALUES” column to reflect the value in the “NAMES” column for each split.
 - Finally, it recombines the columns into a single table on the basis of the ID columns
- We want to do this with the `gapminder` data by using the `obs_type` column for the names, and the `obs_values` column for the values.

```
> gap_normal <- gap_long %>% pivot_wider(names_from = obs_type, values_from = obs_values)
> str(gap_normal)
tibble [1,704 × 6] (S3: tbl_df/tbl/data.frame)
 $ continent: chr [1:1704] "Africa" "Africa" "Africa" "Africa" ...
 $ country  : chr [1:1704] "Algeria" "Algeria" "Algeria" "Algeria" ...
 $ year     : int [1:1704] 1952 1957 1962 1967 1972 1977 1982 1987 1992 1997 ...
 $ pop      : num [1:1704] 9279525 10270856 11000948 12760499 14760787 ...
 $ lifeExp  : num [1:1704] 43.1 45.7 48.3 51.4 54.5 ...
 $ gdpPercap: num [1:1704] 2449 3014 2551 3247 4183 ...
```

- And we can see that this has restored the intermediate form of the data that was so useful to us earlier.
-

Challenge (5min)

- Using `gap_long`, calculate the mean life expectancy, population, and `gdpPercap` for each continent. Hint: use the `group_by()` and `summarize()` functions we learned in the `dplyr` lesson

```
gap_long %>% group_by(continent, obs_type) %>%  
  summarize(means=mean(obs_values))
```



Red sticky for a question or issue



Green sticky if complete



Figure 3: images/red_green_sticky.png